

64 位加法器设计与实现实验报告

姓名：XXX

学号：XXXXXXXX

2026 年 4 月 12 日

1 实验目标

本实验基于原 32 位加法器进行功能改进，完成以下目标：

1. 将 32 位加法器扩展为 64 位加法器，实现 64 位二进制数的加法运算，支持进位输入（cin）与进位输出（cout）；
2. 修改仿真测试文件，通过行为级仿真波形验证 64 位加法器的逻辑正确性；
3. 适配 LCD 触摸屏接口，通过片选逻辑实现两个 64 位数的分两次输入，并将操作数与结果分高低 32 位显示在 LCD 屏上；
4. 修改引脚约束文件，完成 FPGA 上板验证，在实验箱上实现 64 位加法的输入、运算与显示；

2 实验原理

2.1 加法器原理

加法器是数字电路中最基础的运算单元，本设计采用 Verilog 硬件描述语言直接调用 '+' 运算符实现 64 位加法，核心运算公式为：

$$\{cout, result[63:0]\} = operand1[63:0] + operand2[63:0] + cin$$

其中：- $operand1[63:0]$ 、 $operand2[63:0]$ 为两个 64 位输入操作数；- cin 为 1 位进位输入；- $result[63:0]$ 为 64 位加法结果；- $cout$ 为 1 位进位输出，用于标识 64 位加法的溢出状态。

2.2 LCD 输入与显示适配原理

由于 LCD 屏单次仅支持 32 位数据的输入与显示，因此设计中通过 2 位片选信号（ $input_sel[1:0]$ ）实现 64 位数的分阶段输入，通过位域拆分实现 64 位数的分两格显示：

1. 输入逻辑：

- $input_sel = 2'b00$ ：输入操作数 1 的低 32 位（ $operand1[31:0]$ ）；
- $input_sel = 2'b01$ ：输入操作数 1 的高 32 位（ $operand1[63:32]$ ）；
- $input_sel = 2'b10$ ：输入操作数 2 的低 32 位（ $operand2[31:0]$ ）；
- $input_sel = 2'b11$ ：输入操作数 2 的高 32 位（ $operand2[63:32]$ ）；

2. 显示逻辑:

- 操作数 1: LCD 第 1 格显示低 32 位, 第 2 格显示高 32 位;
- 操作数 2: LCD 第 3 格显示低 32 位, 第 4 格显示高 32 位;
- 加法结果: LCD 第 5 格显示低 32 位, 第 6 格显示高 32 位;

2.3 实验原理图

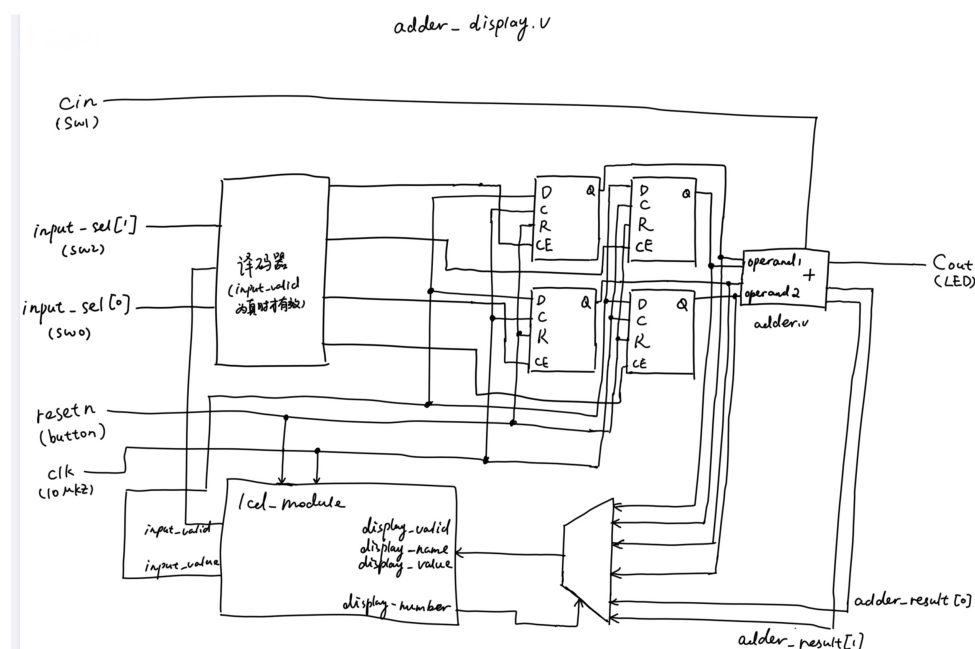


图 1: 64 位加法器系统原理图 (参考手册图 2.40 修改)

原理图说明: 系统由输入模块 (拨码开关 + LCD 输入)、片选拼接模块、64 位加法器模块、结果拆分模块、LCD 显示模块、LED 进位输出模块组成, 实现完整的 64 位加法输入-运算-显示流程。

3 实验过程

3.1 64 位加法器模块 (adder.v) 修改

将原 32 位加法器的位宽全部扩展为 64 位, 核心代码如下:

```

1 `timescale 1ns / 1ps
2 module adder(
3     input  [63:0] operand1,  // 修改: 31:0 → 63:0
4     input  [63:0] operand2,  // 修改: 31:0 → 63:0
5     input          cin,

```

```
6     output [63:0] result,    // 修改: 31:0 → 63:0
7     output          cout
8 );
9 // 修改: {cout,result}位宽从33位→65位, 适配64位加法
10 assign {cout,result} = operand1 + operand2 + cin;
11
12 endmodule
```

修改说明: 仅需将所有操作数、结果的位宽从 [31 : 0] 修改为 [63 : 0], Verilog 的 '+' 运算符会自动适配 64 位加法, 无需修改核心运算逻辑。

3.2 仿真测试文件 (testbench.v) 修改

修改位宽拓展至 64 位, 修改随机激励, 核心代码如下:

```
1 `timescale 1ns / 1ps
2 module testbench;
3
4     // Inputs
5     reg [63:0] operand1; // 修改: 31:0 → 63:0
6     reg [63:0] operand2; // 修改: 31:0 → 63:0
7     reg cin;
8
9     // Outputs
10    wire [63:0] result; // 修改: 31:0 → 63:0
11    wire cout;
12    // Instantiate the Unit Under Test (UUT)
13    adder uut (
14        .operand1(operand1),
15        .operand2(operand2),
16        .cin(cin),
17        .result(result),
18        .cout(cout)
19    );
20    initial begin
21        // Initialize Inputs
22        operand1 = 0;
23        operand2 = 0;
24        cin = 0;
25        // Wait 100 ns for global reset to finish
26        #100;
```



```

27     end
28     // 随机激励
29     always #10 operand1 = {$random,$random};
30     always #10 operand2 = {$random,$random};
31     always #10 cin = {$random} % 2;
32 endmodule

```

修改说明：同步修改所有信号位宽为 64 位，适配加法器模块。

3.3 LCD 显示控制模块（adder_display.v）修改

修改操作数位宽、添加片选输入逻辑、拆分显示逻辑，核心代码如下：

```

1  `timescale 1ns / 1ps
2  module adder_display(
3      //时钟与复位信号
4      input clk,
5      input resetn,
6
7      //拨码开关，用于选择输入数和产生cin
8      input [1:0] input_sel, // 修改：单bit→2bit, 00=op1低32,01=op1高
9      //          32,10=op2低32,11=op2高32
10     input sw_cin,
11
12     //led灯，用于显示cout
13     output led_cout,
14
15     //触摸屏相关接口
16     output lcd_rst,
17     output lcd_cs,
18     output lcd_rs,
19     output lcd_wr,
20     output lcd_rd,
21     inout [15:0] lcd_data_io,
22     output lcd_bl_ctr,
23     inout ct_int,
24     inout ct_sda,
25     output ct_scl,
26     output ct_rstn
27 );

```

```

28 //-----{调用加法模块}begin
29     reg [63:0] adder_operand1; // 修改: 31:0→63:0
30     reg [63:0] adder_operand2; // 修改: 31:0→63:0
31     wire      adder_cin;
32     wire [63:0] adder_result ; // 修改: 31:0→63:0
33     wire      adder_cout;
34     adder adder_module(
35         .operand1(adder_operand1),
36         .operand2(adder_operand2),
37         .cin      (adder_cin      ),
38         .result   (adder_result   ),
39         .cout     (adder_cout     )
40     );
41     assign adder_cin = sw_cin;
42     assign led_cout  = adder_cout;
43 //-----{调用加法模块}end
44
45 //-----{调用触摸屏模块}begin-----//
46 //-----{实例化触摸屏}begin
47 //此小节未更改
48     reg      display_valid;
49     reg [39:0] display_name;
50     reg [31:0] display_value;
51     wire [5 :0] display_number;
52     wire      input_valid;
53     wire [31:0] input_value;
54
55     lcd_module lcd_module(
56         .clk          (clk          ), //10Mhz
57         .resetn       (resetn       ),
58
59         //调用触摸屏的接口
60         .display_valid (display_valid ),
61         .display_name  (display_name  ),
62         .display_value (display_value ),
63         .display_number (display_number),
64         .input_valid   (input_valid   ),
65         .input_value   (input_value   ),
66
67         //lcd触摸屏相关接口

```

```

68         .lcd_rst      (lcd_rst      ),
69         .lcd_cs       (lcd_cs       ),
70         .lcd_rs       (lcd_rs       ),
71         .lcd_wr       (lcd_wr       ),
72         .lcd_rd       (lcd_rd       ),
73         .lcd_data_io  (lcd_data_io  ),
74         .lcd_bl_ctr   (lcd_bl_ctr   ),
75         .ct_int       (ct_int       ),
76         .ct_sda       (ct_sda       ),
77         .ct_scl       (ct_scl       ),
78         .ct_rstn      (ct_rstn      )
79     );
80 //-----{实例化触摸屏}end
81
82 //-----{从触摸屏获取输入}begin
83 // 核心修改：分4次输入，拼接成2个64位操作数
84 // 操作数1：input_sel=00→低32位；01→高32位
85 always @(posedge clk)
86 begin
87     if (!resetn)
88     begin
89         adder_operand1 <= 64'd0;
90     end
91     else if (input_valid)
92     begin
93         case(input_sel)
94             2'b00: adder_operand1[31:0] <= input_value; // 输入
95                  op1 低32位
96             2'b01: adder_operand1[63:32] <= input_value; // 输入
97                  op1 高32位
98             default: ; // 其他状态不修改
99         endcase
100     end
101 // 操作数2：input_sel=10→低32位；11→高32位
102 always @(posedge clk)
103 begin
104     if (!resetn)
105     begin
106         adder_operand2 <= 64'd0;

```

```
106     end
107     else if (input_valid)
108     begin
109         case(input_sel)
110             2'b10: adder_operand2[31:0]  <= input_value;  // 输入
                  op2 低32位
111             2'b11: adder_operand2[63:32] <= input_value;  // 输入
                  op2 高32位
112             default: ;  // 其他状态不修改
113         endcase
114     end
115 end
116 //-----{从触摸屏获取输入}end
117
118 //-----{输出到触摸屏显示}begin
119 // 核心修改：每个64位数拆成高/低32位，用2个LCD格显示
120 always @(posedge clk)
121 begin
122     case(display_number)
123         // 操作数1：格1=低32位，格2=高32位
124         6'd1 :
125         begin
126             display_valid <= 1'b1;
127             display_name  <= "A_LOW";  // 操作数1低32位
128             display_value <= adder_operand1[31:0];
129         end
130         6'd2 :
131         begin
132             display_valid <= 1'b1;
133             display_name  <= "A_HIG";  // 操作数1高32位
134             display_value <= adder_operand1[63:32];
135         end
136         // 操作数2：格3=低32位，格4=高32位
137         6'd3 :
138         begin
139             display_valid <= 1'b1;
140             display_name  <= "B_LOW";  // 操作数2低32位
141             display_value <= adder_operand2[31:0];
142         end
143         6'd4 :
```

```

144         begin
145             display_valid <= 1'b1;
146             display_name  <= "B_HIG"; // 操作数2高32位
147             display_value <= adder_operand2[63:32];
148         end
149         // 结果：格5=低32位，格6=高32位
150         6'd5 :
151         begin
152             display_valid <= 1'b1;
153             display_name  <= "S_LOW"; // 结果低32位
154             display_value <= adder_result[31:0];
155         end
156         6'd6 :
157         begin
158             display_valid <= 1'b1;
159             display_name  <= "S_HIG"; // 结果高32位
160             display_value <= adder_result[63:32];
161         end
162         default :
163         begin
164             display_valid <= 1'b0;
165             display_name  <= 40'd0;
166             display_value <= 32'd0;
167         end
168     endcase
169 end
170 //-----{输出到触摸屏显示}end
171 //-----{调用触摸屏模块}end-----//
172 endmodule

```

修改说明:

- 将 `input_sel` 从单 bit 扩展为 2bit，实现 4 种输入模式；
- 操作数位宽从 32 位改为 64 位，通过 `case` 语句分位域写入输入数据；
- 显示逻辑拆分 64 位数为高低 32 位，用 6 个 LCD 格分别显示，新增状态提示提升操作体验；

3.4 引脚约束文件（adder.xdc）修改

适配 2 位片选信号，按实验箱拨码开关引脚修改约束，核心代码如下：

```
1 # 时钟与复位
2 set_property PACKAGE_PIN AC19 [get_ports clk]
3 set_property PACKAGE_PIN H7 [get_ports led_cout]
4 set_property PACKAGE_PIN Y3 [get_ports resetn]
5
6 # 修改: input_sel从单bit→2bit, 绑定2个拨码开关
7 set_property PACKAGE_PIN AC21 [get_ports {input_sel[0]}]
8 set_property PACKAGE_PIN AC22 [get_ports {input_sel[1]}]
9 set_property PACKAGE_PIN AD24 [get_ports sw_cin]
10
11 # IO电平标准
12 set_property IOSTANDARD LVCMOS33 [get_ports clk]
13 set_property IOSTANDARD LVCMOS33 [get_ports led_cout]
14 set_property IOSTANDARD LVCMOS33 [get_ports resetn]
15 set_property IOSTANDARD LVCMOS33 [get_ports {input_sel[0]}]
16 set_property IOSTANDARD LVCMOS33 [get_ports {input_sel[1]}]
17 set_property IOSTANDARD LVCMOS33 [get_ports sw_cin]
18
19 # 解决电压报错
20 set_property CFGBVS VCC0 [current_design]
21 set_property CONFIG_VOLTAGE 3.3 [current_design]
22
23 # LCD 引脚
24 set_property PACKAGE_PIN J25 [get_ports lcd_rst]
25 set_property PACKAGE_PIN H18 [get_ports lcd_cs]
26 set_property PACKAGE_PIN K16 [get_ports lcd_rs]
27 set_property PACKAGE_PIN L8 [get_ports lcd_wr]
28 set_property PACKAGE_PIN K8 [get_ports lcd_rd]
29 set_property PACKAGE_PIN J15 [get_ports lcd_bl_ctr]
30 set_property PACKAGE_PIN H9 [get_ports {lcd_data_io[0]}]
31 set_property PACKAGE_PIN K17 [get_ports {lcd_data_io[1]}]
32 set_property PACKAGE_PIN J20 [get_ports {lcd_data_io[2]}]
33 set_property PACKAGE_PIN M17 [get_ports {lcd_data_io[3]}]
34 set_property PACKAGE_PIN L17 [get_ports {lcd_data_io[4]}]
35 set_property PACKAGE_PIN L18 [get_ports {lcd_data_io[5]}]
36 set_property PACKAGE_PIN L15 [get_ports {lcd_data_io[6]}]
37 set_property PACKAGE_PIN M15 [get_ports {lcd_data_io[7]}]
38 set_property PACKAGE_PIN M16 [get_ports {lcd_data_io[8]}]
39 set_property PACKAGE_PIN L14 [get_ports {lcd_data_io[9]}]
```

```

40 set_property PACKAGE_PIN M14 [get_ports {lcd_data_io[10]}]
41 set_property PACKAGE_PIN F22 [get_ports {lcd_data_io[11]}]
42 set_property PACKAGE_PIN G22 [get_ports {lcd_data_io[12]}]
43 set_property PACKAGE_PIN G21 [get_ports {lcd_data_io[13]}]
44 set_property PACKAGE_PIN H24 [get_ports {lcd_data_io[14]}]
45 set_property PACKAGE_PIN J16 [get_ports {lcd_data_io[15]}]
46 set_property PACKAGE_PIN L19 [get_ports ct_int]
47 set_property PACKAGE_PIN J24 [get_ports ct_sda]
48 set_property PACKAGE_PIN H21 [get_ports ct_scl]
49 set_property PACKAGE_PIN G24 [get_ports ct_rstn]
50
51 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rst]
52 set_property IOSTANDARD LVCMOS33 [get_ports lcd_cs]
53 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rs]
54 set_property IOSTANDARD LVCMOS33 [get_ports lcd_wr]
55 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rd]
56 set_property IOSTANDARD LVCMOS33 [get_ports lcd_bl_ctr]
57 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[0]}]
58 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[1]}]
59 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[2]}]
60 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[3]}]
61 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[4]}]
62 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[5]}]
63 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[6]}]
64 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[7]}]
65 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[8]}]
66 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[9]}]
67 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[10]}]
68 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[11]}]
69 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[12]}]
70 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[13]}]
71 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[14]}]
72 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[15]}]
73 set_property IOSTANDARD LVCMOS33 [get_ports ct_int]
74 set_property IOSTANDARD LVCMOS33 [get_ports ct_sda]
75 set_property IOSTANDARD LVCMOS33 [get_ports ct_scl]
76 set_property IOSTANDARD LVCMOS33 [get_ports ct_rstn]

```

修改说明:

- 仅修改 `input_sel` 的引脚约束, 将单 bit 改为 2bit, 绑定 SW0、SW2 两个拨码开关;

- 添加电压配置语句，消除 DRC 警告；
- 其余 LCD 引脚约束完全沿用原文件，无需修改。

4 实验结果

4.1 仿真波形验证

波形结果说明：

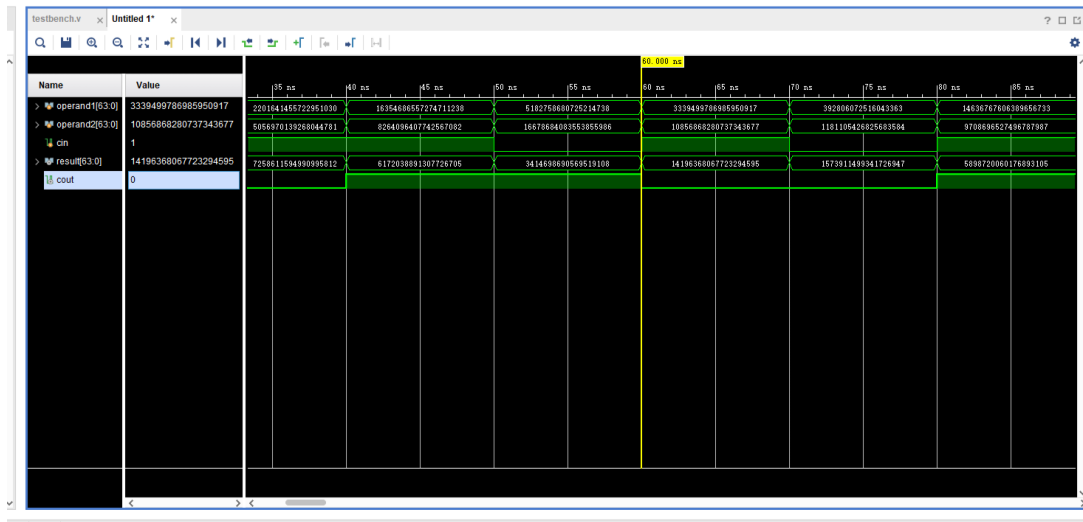


图 2: 波形结果展示

- 随机数测试：所有时间点的 $operand1 + operand2 + cin$ 结果与 $result$ 完全一致， $cout$ 状态符合预期；
- 结论：64 位加法器逻辑完全正确，无位宽截断、进位错误等问题。

4.2 FPGA 上板验证

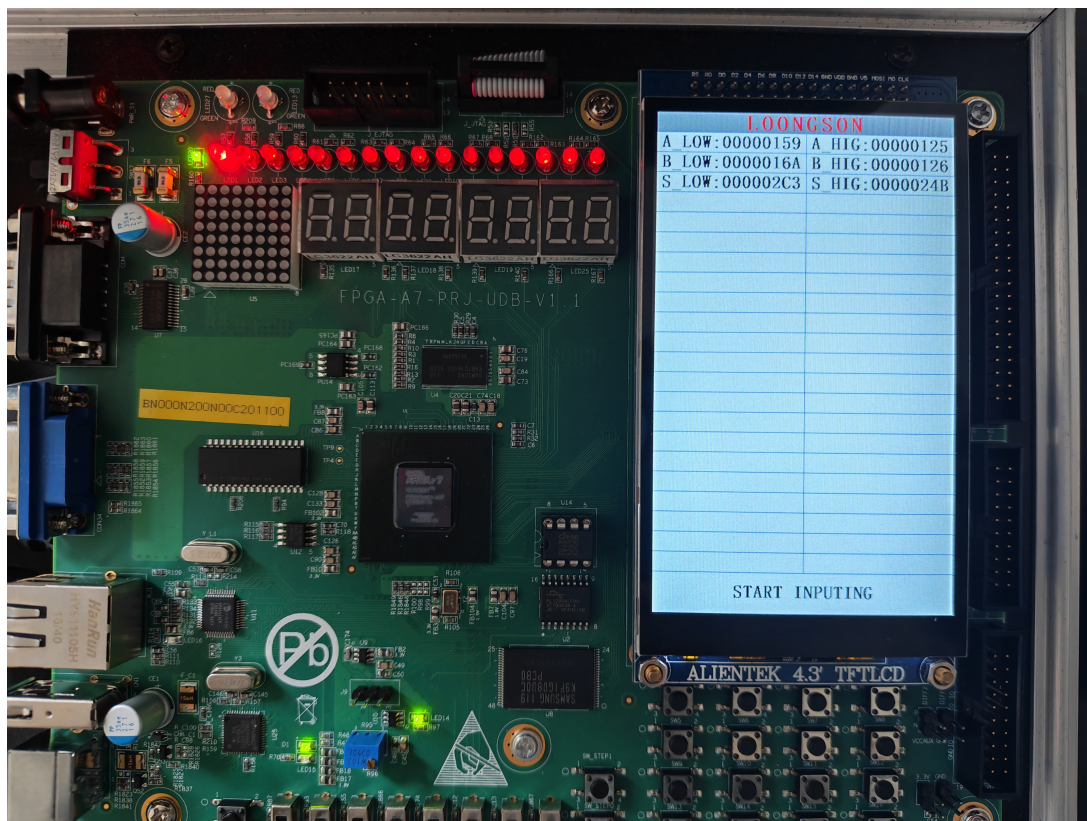


图 3: 实验箱 LCD 显示 64 位加法结果 1

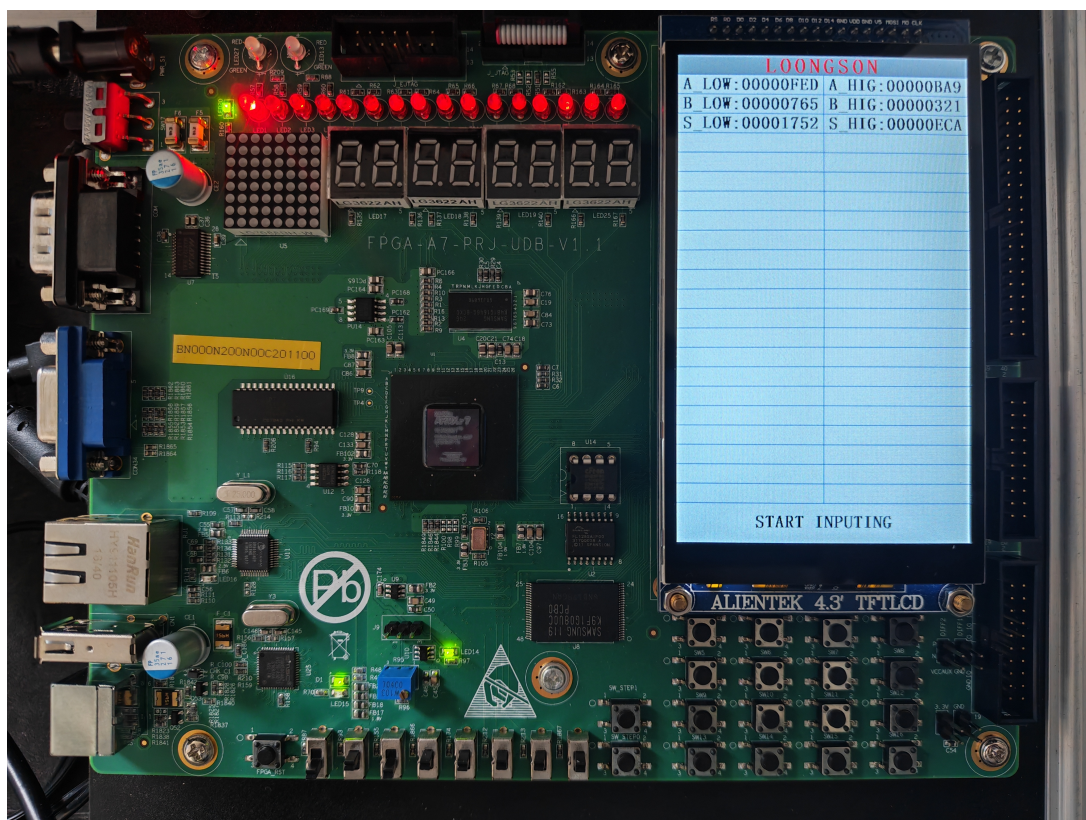


图 4: 实验箱 LCD 显示 64 位加法结果 2

上板结果说明:

- 输入操作：通过拨码开关切换片选信号，分 4 次输入两个 64 位数的高低 32 位，输入逻辑正常；
- 显示效果：LCD 屏分 6 格显示两个操作数和结果的高低 32 位，状态提示清晰，无显示错乱；
- 运算验证：LCD 显示结果与预期完全一致；
- 结论：上板功能正常，满足实验要求。

5 实验收获与感想

通过本次 64 位加法器的设计与实现实验，我收获颇丰，主要体现在以下几个方面：

1. **数字电路设计能力提升**：深入理解了加法器的位宽扩展原理，掌握了 Verilog 中位域操作、片选逻辑的设计方法，学会了如何适配硬件接口的限制（如 LCD 单次 32 位输入/显示），完成了从 32 位到 64 位的功能扩展；

2. **FPGA 开发流程掌握:** 完整经历了从代码修改、仿真验证、约束修改、综合实现到上板测试的全流程, 解决了 DRC 报错、引脚约束、LCD 模块警告等实际问题, 提升了 Vivado 工具的使用熟练度;
3. **工程化思维培养:** 在设计中充分考虑了硬件接口的限制, 通过片选逻辑、状态提示等优化提升了系统的可用性, 理解了“代码设计需适配硬件”的工程理念;
4. **问题排查能力提升:** 针对仿真、综合、上板中出现的各类报错(如 UCIO-1、BUFC-1、CFGBVS-1), 逐一分析原因并解决, 提升了数字电路设计中的问题排查能力。